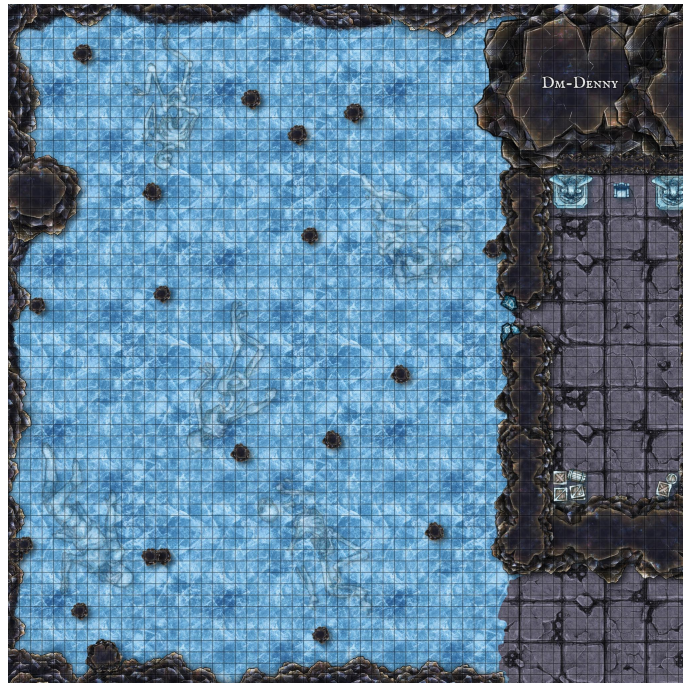


Tugas Kecil 3

Aplikasi Algoritma Pathfinding dalam Penelurusan Solusi dari Permainan *Ice Sliding Puzzle*

IF2211 Strategi Algoritma
Semester 2 Tahun 2025/2026



Disusun oleh:
Muhammad Aufar Rizqi Kusuma

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	3
1.1	Deskripsi Tugas	3
1.2	Spesifikasi Program	4
1.3	Output	4
1.4	Spesifikasi Bonus	6
2	Algoritma Penyelesaian	8
2.1	Algoritma Uniform Cost Search (UCS)	8
2.1.1	Definisi $f(n)$ dan $g(n)$ untuk UCS	8
2.1.2	Implementasi UCS pada Kode Sumber	8
2.1.3	Contoh Pencarian UCS	10
2.2	Algoritma Greedy Best-First Search (GBFS)	11
2.2.1	Definisi $f(n)$ dan $g(n)$ untuk GBFS	11
2.2.2	Implementasi GBFS pada Kode Sumber	12
2.2.3	Contoh Pencarian GBFS	13
2.3	Algoritma A*	14
2.3.1	Definisi $f(n)$ dan $g(n)$ untuk A*	14
2.3.2	Implementasi A* pada Kode Sumber	15
2.3.3	Admissibility pada A*	17
2.3.4	Contoh Pencarian A*	18
2.4	Heuristik yang Digunakan	18
2.4.1	Admissibility Heuristik	19
2.5	Komparasi Teoretis Algoritma Pencarian	20
2.5.1	Kriteria perbandingan	20
2.5.2	Properti Algoritma	20
2.5.3	Perbandingan Teoretis	21
3	Implementasi	23
3.1	Arsitektur Program	23
3.1.1	Lapisan Arsitektur	23
3.1.2	Alur Eksekusi	23
3.1.3	Desain Object-Oriented Programming (OOP)	24
3.2	Direktori Program	26
3.2.1	Penjelasan Direktori Utama	26
3.3	Modul-Modul Program	27
3.3.1	Modul Board	27



3.3.2	Modul Search	28
3.3.3	Modul Rules	29
3.3.4	Modul Heuristic	30
3.3.5	Modul Parser	31
3.3.6	Modul Validator	32
3.3.7	Modul Output	32
3.3.8	Modul Playback	33
3.3.9	Modul Exception	34
3.3.10	Dependency Antar Modul	34
4	Eksperimen	36
4.1	Testing Kasus Dasar	36
4.2	Testing <i>Edge Cases</i>	36
4.3	<i>Post Mortem Analysis</i>	36
5	Penutup	37
6	Lampiran	38
6.1	Tautan GitHub	38
6.2	Checklist Tupil	38

Bab 1

Pendahuluan

1.1 Deskripsi Tugas

Tugas kecil ini berfokus pada pembuatan *solver* untuk permainan *Ice Sliding Puzzle*. Program menerima sebuah papan permainan berbentuk $N \times M$ dari berkas teks, kemudian mencari urutan gerakan yang membawa pin dari posisi awal Z ke tujuan 0. Selama pencarian, pin hanya dapat bergerak ke empat arah utama, dan karena lantai bersifat licin, pin akan terus meluncur sampai tertahan oleh dinding atau batu X.

Selain mencapai tujuan, solusi yang valid juga harus melewati angka-angka pada papan secara berurutan lengkap, mulai dari 0 hingga angka terbesar yang tersedia. Apabila pin melewati lava L, keluar dari batas papan, atau menginjak angka di luar urutan, maka keadaan tersebut dianggap gagal. Dengan demikian, program tidak hanya mencari jalur terpendek atau termurah, tetapi juga jalur yang mematuhi seluruh aturan permainan.

Gambar berikut memperlihatkan contoh papan permainan beserta legenda simbol yang digunakan pada laporan ini.

*	0	*	X	O
*	X	*	*	L
Z	*	1	*	*
*	*	*	X	*

Gambar 1.1: Contoh visualisasi papan *Ice Sliding Puzzle*.

Pada contoh di atas, simbol Z menandai posisi awal, 0 menandai tujuan, X adalah rintangan, L adalah lava, sedangkan 0 dan 1 adalah angka yang harus dilewati secara berurutan sebelum pin dapat dinyatakan menang. Tile bertanda * merepresentasikan jalur kosong yang dapat dilewati pin.

1.2 Spesifikasi Program

- Buatlah sebuah program tersebut dalam bahasa Java, Python, C++, C, atau GO yang mengimplementasikan algoritma Pathfinding (UCS, GBFS, dan A*) untuk mencari solusi dalam permainan Ice Sliding Puzzle Solver.
- Tugas dikerjakan berkelompok dengan anggota maksimal 2 orang, boleh lintas kelas maupun kampus. Silahkan isi pendataan kelompok pada link berikut.
- **Input:** program akan memberikan pengguna sebuah arahan pada terminal (jika tidak mengerjakan bonus) untuk memilih file test case berekstensi .txt, kemudian program membaca file test case tersebut yang berisi konfigurasi awal dari map Permainan yang akan diselesaikan. Perhatikan bahwa input dapat memiliki ukuran papan yang berbeda-beda (tidak harus persegi), serta program harus dapat memvalidasi apakah input yang diberikan merupakan input valid atau bukan.

Contoh perhitungan cost gerakan adalah sebagai berikut.

- Misalkan aktor melakukan satu gerakan ke kanan dan selama gerakan tersebut melewati tiga tile dengan cost berturut-turut: 2, 5, dan 4. Maka cost gerakan itu adalah $2 + 5 + 4 = 11$.
- Total cost solusi adalah jumlah dari semua cost gerakan yang dilakukan sepanjang solusi.
- Perlu diingat: tile 'X' dan 'L' tetap mempunyai nilai cost dalam input, tetapi tidak dihitung dalam solusi valid karena 'X' tidak pernah dilewati dan 'L' menyebabkan game over jika dilewati.

Contoh:

Gerakan 1: tiles traversed costs = 2, 5, 4, maka movement cost = 11

Gerakan 2: tiles traversed costs = 3, 2, maka movement cost = 5

Total cost solusi = $11 + 5 = 16$

1.3 Output

Program harus menghasilkan keluaran berikut setelah pencarian selesai:

1. Menampilkan solusi gerakan yang ditemukan (mis. RULUDR...).
2. Menampilkan total cost dari solusi.
3. Menampilkan visualisasi papan pada solusi, yaitu keadaan papan sehingga aktor dapat mencapai titik 0 menggunakan algoritma pathfinding yang dipilih. Visualisasi ini harus menandai lintasan akhir yang ditempuh aktor.
4. Menampilkan jumlah konfigurasi / iterasi yang ditinjau oleh algoritma. Program harus menyediakan opsi untuk menyimpan setiap iterasi (state) ke file berekstensi .txt sehingga dapat dianalisis atau diputar ulang nanti.

5. Menampilkan waktu eksekusi algoritma dalam milidetik. Pengukuran waktu harus meliputi hanya waktu komputasi (search), tidak termasuk waktu I/O membaca/menulis file.
6. Playback (bukan live update), dimana setelah algoritma selesai, program harus dapat memvisualisasikan proses pathfinding sebagai rangkaian langkah yang dapat diputar mundur/maju. Contoh playback di CLI dapat berisi fitur-fitur di bawah ini.
 - (a) Arrow kiri/kanan untuk mundur/maju satu step.
 - (b) ESC: prompt user untuk memasukkan nomor step X dan langsung lompat ke step tersebut.

Catatan: untuk mode GUI (bonus), playback harus memiliki kontrol kecepatan (play/pause/seek) dan kemampuan maju/mundur yang setara dengan pengalaman video. Berikut adalah contoh file .txt yang akan dijadikan sebagai input.

Baris pertama memuat 2 angka (N, M) yang menyatakan panjang dan lebar papan
N baris berikutnya merepresentasikan papan
N baris berikutnya merepresentasikan cost untuk melewati tile tersebut

Contoh Input

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X***1LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

Keterangan:

- * = path yang bisa dilewati.
- X = Rintangan/Batu. Aktor akan berhenti tepat sebelum batu.
- L = Lava. Melewati lava akan mengalami game over (meskipun tidak berhenti tepat di lava).

- Z = Aktor/Pengguna.
- 0 = Titik tujuan.
- $\langle i \rangle$ = Angka. Ini adalah modifikasi pada spesifikasi tugas. Aktor harus melewati angka-angka ini sesuai urutan yang ada pada papan. Petak berangka 1 harus dilewati setelah aktor melewati petak berangka 0. petak berangka 2 harus dilewati setelah petak berangka 1 dan seterusnya (maksimal sampai 9).

Aturan tambahan yang penting.

- Jika aktor menginjak angka di luar urutan (mis. menginjak 1 sebelum 0), maka kondisi tersebut dianggap game over.
- Setelah sebuah petak angka dilewati sesuai urutan, petak tersebut berubah menjadi petak biasa dan dapat dilalui lagi tanpa batasan urutan.
- Petak angka adalah tile normal (bukan rintangan). Akibatnya, aktor akan meluncur melewatinya jika bergerak tanpa berhenti.
- Jika aktor bergerak menuju sisi papan (kanan/atas/kiri/bawah) dan tidak ada rintangan pada sisi tersebut untuk menghentikannya, maka aktor akan keluar papan dan kondisi dianggap game over. Dengan demikian, permainan harus diulang dari awal.
- Pin/aktor harus tepat berhenti pada titik 0 dan semua angka harus telah dilewati sesuai urutan agar kondisi dianggap sukses.
- Setiap tile memiliki cost traversal yang diberikan pada bagian kedua input. Cost merepresentasikan biaya ketika aktor melewati suatu tile.
- Cost gerakan dihitung dari jumlah cost seluruh tile yang dilalui selama satu gerakan sliding. Tile posisi awal pada awal gerakan tidak dihitung, tetapi tile tempat aktor berhenti pada akhir gerakan tetap dihitung.

1.4 Spesifikasi Bonus

Poin maksimal untuk bonus adalah 10. Mahasiswa dapat memilih satu atau beberapa spesifikasi bonus di bawah ini; total poin tambahan yang diberikan untuk tugas tidak boleh melebihi 10 poin.

1. **GUI penuh untuk program (maks 6 poin).** Membuat antarmuka grafis yang memungkinkan pengguna memilih file input, menampilkan papan, mengatur kecepatan playback, dan memutar langkah mundur/maju. Kriteria penilaian:
2. **Tambahkan algoritma pathfinding tambahan (maks 2 poin).** Implementasi setidaknya satu algoritma pathfinding lain selain yang diwajibkan (mis. Dijkstra, IDA*, BFS adaptif untuk sliding puzzle, atau algoritma lain yang relevan). Penilaian:

3. **Tambahkan dua alternatif heuristik (maks 2 poin).** Sediakan dua heuristik tambahan untuk A^* selain heuristik utama. Kriteria:
4. **Optimasi kecepatan pencarian (maks 5 poin, tetapi total bonus tetap dibatasi).** Optimasi yang dapat meningkatkan kecepatan menemukan solusi. Penilaian didasarkan pada bukti eksperimen dan metrik yang jelas.

Catatan penting tentang pemberian nilai.

- Total poin bonus yang diberikan tidak boleh melebihi 10, meskipun jumlah maksimum per-item jika dijumlahkan lebih besar. Dosen/TA akan memilih kombinasi fitur yang memberi nilai maksimal hingga 10 poin.
- Untuk setiap fitur bonus, sertakan bukti fungsionalitas: screenshot GUI atau rekaman pendek (untuk GUI), hasil benchmark (untuk kecepatan), dan contoh keluaran atau log yang menunjukkan algoritma/heuristik baru bekerja.
- Semua kode bonus harus disertakan pada repositori akhir dan dilengkapi instruksi build/run di README.

Instruksi pengumpulan bonus.

- Sertakan penjelasan singkat di README tentang fitur bonus yang diimplementasikan dan bagaimana mengaktifkannya (mis. argumen CLI atau flag pada GUI).
- Apabila mengumpulkan GUI, sertakan screenshot dan langkah untuk menjalankan (dependensi, command build, dan cara membuka file input).

Bab 2

Algoritma Penyelesaian

2.1 Algoritma Uniform Cost Search (UCS)

Uniform Cost Search (UCS) adalah algoritma pencarian yang mengembangkan node dengan biaya jalur terkecil $g(n)$ terlebih dahulu. UCS menjamin menemukan solusi dengan total biaya minimum apabila semua biaya tepi bernilai tidak negatif, karena selalu memilih state dengan g terkecil untuk diekspansi [1, 2]. Beberapa sifat penting UCS adalah sebagai berikut.

- **Optimalitas.** UCS optimal jika biaya tiap langkah non-negatif.
- **Keterlengkapan.** UCS akan menemukan solusi apabila ruang status berhingga dan ada path menuju tujuan.
- **Kompleksitas.** Bergantung pada branching factor dan distribusi biaya. Pada kasus terburuk dapat menimbulkan eksplorasi besar jika banyak node memiliki biaya jalur serupa.

2.1.1 Definisi $f(n)$ dan $g(n)$ untuk UCS

Dalam implementasi solver Ice Sliding Puzzle ini, definisi fungsi biaya yang digunakan adalah sebagai berikut.

- $g(n)$: total biaya kumulatif dari semua gerakan yang dilakukan dari keadaan awal sampai mencapai node/state n . Untuk setiap gerakan sliding, biaya gerakan adalah jumlah cost dari tiap tile yang dilalui selama sliding (catatan: tile posisi awal sebelum sliding tidak dihitung, tetapi tile tempat aktor berhenti dihitung).
- $f(n)$: untuk UCS, kita menggunakan $f(n) = g(n)$. Prioritas antrian eksplorasi didasarkan pada nilai $g(n)$ terkecil.

2.1.2 Implementasi UCS pada Kode Sumber

Berdasarkan implementasi di `src/core/search/Search.cpp`, UCS dijalankan melalui fungsi umum `Search::solve` dengan aturan prioritas `computePriority =`

`gCost`. State direpresentasikan sebagai triple (`row`, `col`, `nextCheckpoint`). Di bawah ini adalah uraian langkah demi langkah yang merepresentasikan apa yang dilakukan kode pada setiap tahap.

1. **Inisialisasi state awal:** buat node awal dengan posisi `start`, `nextCheckpoint=0`, `g=0`, dan hitung heuristik (meskipun untuk UCS heuristik tidak dipakai untuk prioritas, nilai `h` biasanya disimpan untuk konsistensi struktur node).
2. **Siapkan frontier:** buat `priority_queue` (disebut `frontier`) yang mengurutkan node menurut prioritas; untuk UCS prioritas ini adalah `gCost`.
3. **Best-cost map:** buat `unordered_map<StateKey,int>` bernama `bestCost` untuk menyimpan biaya terbaik (terkecil) yang pernah ditemukan untuk setiap state (`key = posisi + nextCheckpoint`).
4. **Masukkan start:** push node awal ke `frontier` dan set `bestCost[start.state] = 0`.
5. **Main loop:** selama `frontier` tidak kosong, ambil node dengan prioritas terkecil (`pop`): ini adalah node dengan `g` terkecil pada saat itu.
6. **Pruning cepat:** jika node yang di-`pop` memiliki `g > bestCost[state]` berarti ada jalur lebih baik yang sudah dicatat -> lewati node tersebut (`continue`).
7. **Cek goal:** periksa apakah node memenuhi kondisi goal (`posisi = goal` dan `nextCheckpoint > maxCheckpoint`). Jika ya, lakukan rekonstruksi jalur (`back-track` menggunakan `parentIndex`) dan kembalikan hasil.
8. **Generate successor:** untuk setiap arah gerak (`Up/Down/Left/Right`) panggil `rules.applyMove(board, current.pos, current.nextCheckpoint, dir)`. Fungsi ini meng-"slide" aktor sampai berhenti, mengakumulasi biaya tile yang dilalui, dan mengembalikan posisi akhir, biaya gerak serta nilai `nextCheckpoint` baru bila melewati checkpoint.
9. **Filter outcome tidak valid:** jika hasil `applyMove` menyatakan `outOfBounds`, `hitLava`, atau gerakan invalid lainnya, abaikan successor tersebut.
10. **Hitung biaya kumulatif:** untuk setiap successor valid, hitung `next.g = current.g + outcome.moveCost` dan bentuk `next.state = (outcome.end, outcome.nextCheckpoint)`.
11. **Pruning berdasarkan bestCost:** jika `bestCost[next.state]` ada dan `next.g >= bestCost[next.state]`, abaikan successor (jalur lebih mahal atau sama sudah ada). Jika lebih baik, set `bestCost[next.state] = next.g`.
12. **Enqueue:** masukkan successor ke `frontier` dengan prioritas `next.g`. Simpan indeks `parent` untuk rekonstruksi jalur.

Algorithm 1 Uniform Cost Search (UCS) pada Solver

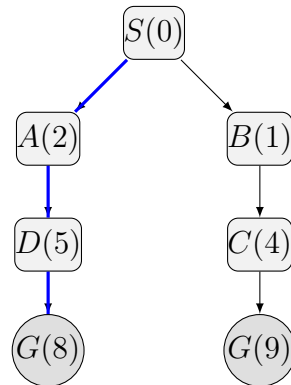
Require: *board, rules*

Ensure: *SearchResult*

```
1: start  $\leftarrow$  (board.start, nextCheckpoint = 0, g = 0)
2: frontier  $\leftarrow$  priority queue terurut naik berdasarkan g
3: masukkan start ke frontier
4: bestCost[(start.pos, start.nextCheckpoint)]  $\leftarrow$  0
5: while frontier tidak kosong do
6:   current  $\leftarrow$  pop(frontier)
7:   if current.g > bestCost[current.state] then
8:     continue
9:   end if
10:  if isGoalState(current.pos, current.nextCheckpoint) then
11:    return reconstructPath(current)
12:  end if
13:  for setiap dir  $\in$  {Up, Down, Left, Right} do
14:    outcome  $\leftarrow$  applyMove(board, current.pos, current.nextCheckpoint, dir)
15:    if outcome.invalid or outcome.hitLava or outcome.outOfBounds then
16:      continue
17:    end if
18:    next.g  $\leftarrow$  current.g + outcome.moveCost
19:    next.state  $\leftarrow$  (outcome.end, outcome.nextCheckpoint)
20:    if outcome.end = board.goal and outcome.nextCheckpoint  $\leq$ 
      board.maxCheckpoint then
21:      continue
22:    end if
23:    if next.state  $\in$  bestCost and next.g  $\geq$  bestCost[next.state] then
24:      continue
25:    end if
26:    bestCost[next.state]  $\leftarrow$  next.g
27:    masukkan next ke frontier dengan prioritas next.g
28:  end for
29: end while
30: return failure
```

2.1.3 Contoh Pencarian UCS

Sebagai ilustrasi, misalkan terdapat graf pencarian sederhana berikut. Angka di dalam kurung menunjukkan biaya kumulatif $g(n)$.



Gambar 2.1: Contoh pohon pencarian UCS. Jalur terpilih adalah $S \rightarrow A \rightarrow D \rightarrow G$ karena memiliki biaya kumulatif minimum pada setiap ekspansi.

Pada contoh ini, urutan rute yang dipilih adalah $S \rightarrow A \rightarrow D \rightarrow G$. UCS selalu memilih node dengan $g(n)$ terkecil, sehingga meskipun ada cabang lain yang tampak lebih pendek secara langkah, jalur dengan biaya kumulatif paling kecil tetap diprioritaskan.

2.2 Algoritma Greedy Best-First Search (GBFS)

Greedy Best-First Search (GBFS) merupakan algoritma pencarian yang memilih node untuk diekspansi berdasarkan nilai heuristik $h(n)$ saja, yakni node yang dianggap paling "dekat" ke tujuan menurut heuristik. GBFS tidak mempertimbangkan biaya jalur $g(n)$ sehingga dapat menjadi sangat cepat namun tidak menjamin optimalitas atau keterlengkapan dalam semua kasus [2]. Karakteristik GBFS dapat dijabarkan sebagai berikut.

- **Kecepatan.** Seringkali lebih cepat menemukan solusi dibanding pencarian yang mempertimbangkan $g(n)$, terutama bila heuristik sangat informatif.
- **Non-optimal.** GBFS tidak menjamin menemukan solusi dengan biaya minimal.
- **Sensitif terhadap heuristik.** kualitas heuristik sangat menentukan performa dan hasil pencarian.

2.2.1 Definisi $f(n)$ dan $g(n)$ untuk GBFS

Untuk GBFS pada solver ini berlaku

- $g(n)$: sama seperti pada UCS, dapat dihitung sebagai total biaya kumulatif sampai node n , namun nilai ini *tidak* digunakan untuk menentukan urutan ekspansi.
- $f(n)$: GBFS menggunakan $f(n) = h(n)$, yaitu hanya mengurutkan node menurut estimasi heuristik menuju tujuan. Dengan demikian antrian prioritas berdasarkan $h(n)$ saja.

2.2.2 Implementasi GBFS pada Kode Sumber

Pada kode sumber yang sama, GBFS tidak membuat fungsi terpisah; perbedaannya hanya pada prioritas antrian: `computePriority = hCost`. Nilai heuristik dihitung oleh `Heuristic::estimate (src/core/heuristic/Heuristic.cpp)` sesuai pilihan `HeuristicType`. Di bawah ini alur langkah demi langkah implementasi GBFS dalam kode.

1. **Inisialisasi:** buat node awal dengan `g=0` dan hitung `h = Heuristic::estimate(...)` untuk posisi awal.
2. **Frontier:** buat `priority_queue` yang mengurutkan node berdasarkan nilai heuristik `h` (nilai lebih kecil = prioritas lebih tinggi).
3. **Best-cost map:** seperti UCS, tetap gunakan `bestCost` untuk menyimpan `g` terbaik yang ditemukan untuk setiap state. Ini membantu menghindari revisits yang jelas lebih mahal.
4. **Masukkan start:** push node awal ke frontier dengan prioritas `start.h`.
5. **Main loop:** pop node dengan `h` terkecil.
6. **Pruning:** jika `current.g > bestCost[current.state]` maka lewati.
7. **Cek goal:** jika memenuhi kondisi goal, rekonstruksi dan kembalikan jalur.
8. **Generate successor:** untuk tiap arah, panggil `rules.applyMove` untuk mendapatkan posisi akhir dan `moveCost` sama seperti pada UCS.
9. **Validasi:** tolak successor yang `outOfBounds`, `hitLava`, atau `invalid`.
10. **Kalkulasi g dan h:** hitung `next.g = current.g + outcome.moveCost` dan `next.h = Heuristic::estimate(..., outcome.end, outcome.nextCheckpoint)`.
11. **Pruning bestCost:** jika ada entri `bestCost[next.state]` yang lebih baik, abaikan; jika lebih baik, perbarui `bestCost[next.state] = next.g`.
12. **Enqueue:** push successor ke frontier dengan prioritas `next.h`. Simpan parent index untuk rekonstruksi.

Catatan implementasi khusus GBFS di kode:

- Meski prioritas adalah heuristik, penyimpanan `g` dan penggunaan `bestCost` membantu menghindari eksplorasi ulang yang jelas lebih mahal.
- Performa sangat bergantung pada kualitas heuristik: heuristik yang informatif mengarahkan eksplorasi ke goal dengan cepat.

Algorithm 2 Greedy Best-First Search (GBFS) pada Solver

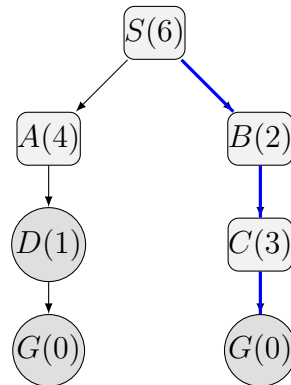
Require: *board, rules, heuristicType*

Ensure: *SearchResult*

```
1: start.g  $\leftarrow 0$ 
2: start.h  $\leftarrow \text{estimate}(\text{board}, \text{start.pos}, \text{start.nextCheckpoint}, \text{heuristicType})$ 
3: frontier  $\leftarrow$  priority queue terurut naik berdasarkan h
4: masukkan start ke frontier dengan prioritas start.h
5: bestCost[(start.pos, start.nextCheckpoint)]  $\leftarrow 0$ 
6: while frontier tidak kosong do
7:   current  $\leftarrow \text{pop}(\text{frontier})$ 
8:   if current.g > bestCost[current.state] then
9:     continue
10:  end if
11:  if isGoalState(current.pos, current.nextCheckpoint) then
12:    return reconstructPath(current)
13:  end if
14:  for setiap dir  $\in \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$  do
15:    outcome  $\leftarrow \text{applyMove}(\text{board}, \text{current.pos}, \text{current.nextCheckpoint}, \text{dir})$ 
16:    if outcome.invalid or outcome.hitLava or outcome.outOfBounds then
17:      continue
18:    end if
19:    next.g  $\leftarrow \text{current.g} + \text{outcome.moveCost}$ 
20:    next.h  $\leftarrow \text{estimate}(\text{board}, \text{outcome.end}, \text{outcome.nextCheckpoint}, \text{heuristicType})$ 
21:    next.state  $\leftarrow (\text{outcome.end}, \text{outcome.nextCheckpoint})$ 
22:    if outcome.end = board.goal and outcome.nextCheckpoint  $\leq$ 
      board.maxCheckpoint then
23:      continue
24:    end if
25:    if next.state  $\in$  bestCost and next.g  $\geq$  bestCost[next.state] then
26:      continue
27:    end if
28:    bestCost[next.state]  $\leftarrow \text{next.g}$ 
29:    masukkan next ke frontier dengan prioritas next.h
30:  end for
31: end while
32: return failure
```

2.2.3 Contoh Pencarian GBFS

Misalkan heuristik yang digunakan memberi estimasi jarak ke tujuan seperti nilai di bawah ini. Angka dalam kurung adalah $h(n)$.



Gambar 2.2: Contoh pohon pencarian GBFS. Jalur terpilih adalah $S \rightarrow B \rightarrow C \rightarrow G$ karena node-node pada cabang ini memiliki nilai heuristik paling kecil.

Pada contoh ini, GBFS memilih node dengan nilai heuristik terkecil terlebih dahulu, sehingga jalur yang diambil tidak selalu jalur termurah. Fokus utamanya adalah mendekati tujuan secepat mungkin menurut estimasi heuristik.

2.3 Algoritma A*

A* (A-star) merupakan algoritma *Pathfinding* yang menggabungkan keuntungan UCS dan heuristik dengan menggunakan fungsi evaluasi

$$f(n) = g(n) + h(n),$$

untuk melakukan memilih path dengan cost paling minimum. Dalam hal ini, $g(n)$ adalah biaya jalur dari root ke node n dan $h(n)$ adalah heuristik estimasi biaya dari n ke tujuan. A* bersifat optimal jika heuristik h adalah *admissible* (tidak melebihi cost sebenarnya) dan akan efisien apabila h juga *consistent* (monotonik) [3, 2]. Sifat-sifat A* adalah sebagai berikut.

- **Optimalitas.** Terjamin bila h *admissible*.
- **Efisiensi.** Jumlah node yang diekspansi seminimal mungkin di antara semua algoritma yang menggunakan heuristik yang sama dan tetap optimal.
- **Ketergantungan heuristik.** Heuristik yang lebih informatif (mendekati nilai cost sebenarnya) akan mengurangi ruang pencarian.

2.3.1 Definisi $f(n)$ dan $g(n)$ untuk A*

Pada A* yang diimplementasikan untuk Ice Sliding Puzzle adalah sebagai berikut.

- $g(n)$: sama dengan UCS, total biaya kumulatif dari seluruh gerakan sampai node n , dengan tiap gerakan bernilai jumlah cost tile yang dilintasi selama sliding.

- $f(n)$: $f(n) = g(n) + h(n)$ di mana $h(n)$ adalah heuristik estimasi biaya tersisa. Untuk memastikan A^* tetap optimal, pilih heuristik yang *admissible*. Contoh heuristik *admissible* yang sering dipakai pada grid-based sliding problems adalah

$$h(n) = d_{\text{manhattan}}(\text{pos}(n), \text{goal}) \times c_{\min},$$

di mana $d_{\text{manhattan}}$ adalah jarak Manhattan antara posisi berhenti aktor pada state n dan posisi tujuan, dan $c_{\min}$ adalah lower bound dari cost tiap tile pada instance (mis. nilai minimal dari matriks cost). Karena setiap langkah sliding setidaknya akan menempuh jarak fisik minimal dan setiap tile memiliki biaya tidak kurang dari $c_{\min}$, heuristik ini tidak melebihi biaya sebenarnya sehingga *admissible*.

2.3.2 Implementasi A^* pada Kode Sumber

A^* juga berbagi kerangka yang sama di `Search::solve`, tetapi prioritasnya menggunakan jumlah biaya aktual dan heuristik: `computePriority = gCost + hCost`. Di bawah ini langkah demi langkah yang persis tercermin di implementasi kode.

1. **Inisialisasi node awal:** buat node awal dengan `g=0` dan hitung `h = Heuristic::estimate(...)` berdasarkan `HeuristicType` yang dipilih.
2. **Frontier:** buat `priority_queue` yang mengurutkan node berdasarkan nilai `f = g + h` (nilai lebih kecil = prioritas lebih tinggi). Comparator di kode juga menerapkan tie-breaker tambahan (mis. prioritas, lalu `gCost`, lalu urutan masuk) untuk determinisme.
3. **Best-cost map:** buat atau gunakan `bestCost` untuk menyimpan `g` terbaik per state. Ini memastikan kita tidak memproses jalur yang jelas lebih mahal.
4. **Masukkan start:** push node awal ke frontier dengan prioritas `start.g + start.h`.
5. **Main loop:** pop node dengan fungsi evaluasi `f` terkecil.
6. **Pruning cepat:** jika `current.g > bestCost[current.state]` lewati node.
7. **Cek goal:** jika kondisi goal terpenuhi (posisi = goal dan semua checkpoint telah dilewati), rekonstruksi jalur dan kembalikan hasil.
8. **Generate successor:** panggil `rules.applyMove` untuk tiap arah gerak. Fungsi ini memberikan posisi akhir, biaya gerak, dan update `nextCheckpoint` jika checkpoint dilintasi.
9. **Validasi outcome:** abaikan successor jika `outcome.invalid`, `outcome.hitLava`, atau `outcome.outOfBounds`.
10. **Kalkulasi g, h, f:** hitung `next.g = current.g + outcome.moveCost`, lalu `next.h = Heuristic::estimate(..., outcome.end, outcome.nextCheckpoint)`; prioritas yang akan dimasukkan adalah `next.f = next.g + next.h`.

11. **Pruning bestCost**: jika sudah ada `bestCost[next.state]` dengan nilai lebih kecil atau sama, abaikan; jika lebih kecil, perbarui entry tersebut.
12. **Enqueue**: push successor ke frontier dengan prioritas `next.f` dan simpan parent untuk rekonstruksi.

Catatan implementasi penting:

- **StateKey** yang memuat `nextCheckpoint` membuat posisi yang sama dengan progres berbeda menjadi state berbeda ini penting untuk permasalahan checkpoint ordering.
- Pruning pada `bestCost` menggunakan `gCost` menjaga agar A* tidak memproses jalur yang lebih mahal menuju state yang sama, sekaligus kompatibel dengan heuristik admissible untuk memastikan optimalitas.
- Penentuan nilai heuristik (admissible vs. informed) menentukan apakah A* yang berjalan akan optimal dan seberapa efisien ia akan memotong ruang pencarian.

Algorithm 3 A* Search pada Solver

Require: *board, rules, heuristicType*

Ensure: *SearchResult*

```
1: start.g  $\leftarrow$  0
2: start.h  $\leftarrow$  estimate(board, start.pos, start.nextCheckpoint, heuristicType)
3: frontier  $\leftarrow$  priority queue terurut naik berdasarkan (g + h)
4: masukkan start ke frontier dengan prioritas start.g + start.h
5: bestCost[(start.pos, start.nextCheckpoint)]  $\leftarrow$  0
6: while frontier tidak kosong do
7:   current  $\leftarrow$  pop(frontier)
8:   if current.g > bestCost[current.state] then
9:     continue
10:  end if
11:  if isGoalState(current.pos, current.nextCheckpoint) then
12:    return reconstructPath(current)
13:  end if
14:  for setiap dir  $\in$  {Up, Down, Left, Right} do
15:    outcome  $\leftarrow$  applyMove(board, current.pos, current.nextCheckpoint, dir)
16:    if outcome.invalid or outcome.hitLava or outcome.outOfBounds then
17:      continue
18:    end if
19:    next.g  $\leftarrow$  current.g + outcome.moveCost
20:    next.h  $\leftarrow$  estimate(board, outcome.end, outcome.nextCheckpoint, heuristicType)
21:    next.state  $\leftarrow$  (outcome.end, outcome.nextCheckpoint)
22:    if outcome.end = board.goal and outcome.nextCheckpoint  $\leq$ 
      board.maxCheckpoint then
23:      continue
24:    end if
25:    if next.state  $\in$  bestCost and next.g  $\geq$  bestCost[next.state] then
26:      continue
27:    end if
28:    bestCost[next.state]  $\leftarrow$  next.g
29:    masukkan next ke frontier dengan prioritas next.g + next.h
30:  end for
31: end while
32: return failure
```

2.3.3 Admissibility pada A*

Secara teori, sebuah heuristik $h(n)$ disebut *admissible* jika untuk setiap node n berlaku

$$0 \leq h(n) \leq h^*(n),$$

dengan $h^*(n)$ adalah biaya minimum sebenarnya dari n ke goal. Artinya, heuristik tidak boleh *overestimate* biaya sisa. Jika A* menggunakan heuristik *admissible*, maka solusi pertama yang mencapai goal dijamin optimal [3, 2].

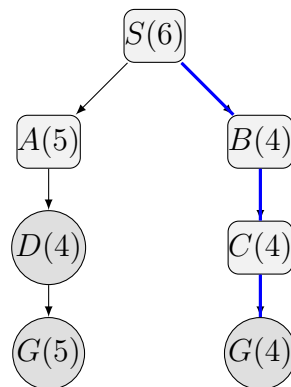
Selain admissible, sering dipakai juga syarat *consistency* (monotonik), dimana

$$h(n) \leq c(n, n') + h(n'),$$

untuk setiap transisi dari n ke tetangga n' . Heuristik yang consistent selalu admissible, dan membuat nilai $f(n)$ tidak menurun sepanjang lintasan sehingga implementasi A^* lebih stabil.

2.3.4 Contoh Pencarian A^*

Pada ilustrasi berikut, angka di dalam kurung menunjukkan nilai $f(n) = g(n) + h(n)$.



Gambar 2.3: Contoh pohon pencarian A^* . Jalur terpilih adalah $S \rightarrow B \rightarrow C \rightarrow G$ karena kombinasi biaya aktual $g(n)$ dan heuristik $h(n)$ menghasilkan nilai $f(n)$ terkecil.

Pada contoh ini, A^* tidak hanya melihat kedekatan ke tujuan, tetapi juga biaya yang sudah ditempuh. Karena itu, jalur yang dipilih bisa berbeda dari GBFS dan lebih terarah menuju solusi optimal dibandingkan pencarian berbasis heuristik murni.

2.4 Heuristik yang Digunakan

Pada implementasi solver ini, tersedia beberapa heuristik yang dapat dipilih untuk dipakai pada algoritma A^* maupun GBFS. Masing-masing heuristik dirancang untuk memberi estimasi jarak atau biaya yang berbeda terhadap posisi tujuan.

1. ManhattanToTarget

Heuristik ini menghitung jarak Manhattan dari posisi aktor saat ini ke titik tujuan 0. Jika posisi aktor adalah (x_1, y_1) dan tujuan berada pada (x_2, y_2) , maka

$$h(n) = |x_1 - x_2| + |y_1 - y_2|.$$

Heuristik ini sederhana dan cepat dihitung, sehingga cocok sebagai estimasi awal. Untuk A^* , nilai ini dapat dikalikan dengan biaya minimum tile agar lebih sesuai dengan fungsi cost.

2. RemainingPlusManhattan

Heuristik ini menggabungkan dua komponen, yaitu jumlah checkpoint/angka yang masih tersisa untuk dilewati dan jarak Manhattan ke target berikutnya. Secara intuitif, semakin banyak angka yang belum dilewati, semakin besar nilai heuristiknya. Bentuk sederhananya dapat ditulis sebagai

$$h(n) = \alpha \cdot r(n) + \beta \cdot d_{\text{manhattan}}(\text{pos}(n), \text{target}(n)),$$

dengan $r(n)$ adalah jumlah checkpoint yang tersisa, $\text{target}(n)$ adalah target yang harus dicapai berikutnya, dan α, β adalah bobot pengali.

3. WeightedRemainingToGoal

Heuristik ini memberi bobot lebih besar pada sisa checkpoint dan jarak menuju tujuan akhir. Dibandingkan heuristic sebelumnya, pendekatan ini lebih agresif karena mendorong pencarian untuk segera menyelesaikan urutan checkpoint sebelum fokus penuh ke tujuan akhir. Bentuk umumnya adalah

$$h(n) = \gamma \cdot r(n) + \delta \cdot d_{\text{manhattan}}(\text{pos}(n), \text{goal}),$$

dengan γ dan δ sebagai bobot. Heuristik ini berguna jika ingin pencarian lebih terarah, tetapi perlu diuji agar tetap memberikan perilaku yang stabil pada berbagai papan.

Secara umum, heuristik pertama lebih ringan dan sederhana, sedangkan dua heuristik lainnya mencoba memasukkan informasi progres permainan, yaitu jumlah checkpoint yang belum dilewati. Pemilihan heuristik bergantung pada tujuan pencarian, apakah lebih mengutamakan kecepatan komputasi, jumlah node yang diekspansi, atau kualitas solusi yang ditemukan.

2.4.1 Admissibility Heuristik

- **ManhattanToTarget:** pada bentuk yang dipakai sebagai estimasi jarak murni, heuristik ini dapat dianggap *admissible* selama ia hanya memberikan batas bawah terhadap biaya sisa. Dalam implementasi yang memperhitungkan cost tile, bentuk yang paling aman adalah $d_{\text{manhattan}}(\text{pos}(n), \text{goal}) \times c_{\text{min}}$, dengan c_{min} sebagai cost tile terkecil pada papan. Jika hanya memakai Manhattan tanpa pengali cost minimum, heuristik tersebut tetap cenderung menjadi batas bawah pada jumlah langkah, tetapi belum tentu merepresentasikan batas bawah cost secara ketat pada papan dengan bobot tile yang bervariasi.
- **RemainingPlusManhattan:** heuristik ini **tidak dijamin admissible**. Alasannya, komponen jumlah checkpoint tersisa $r(n)$ dan bobot α, β dapat membuat estimasi lebih besar dari biaya sisa yang sebenarnya. Begitu heuristik mulai menambahkan penalti untuk checkpoint secara agresif, nilai $h(n)$ berpotensi melebihi cost minimum menuju solusi.
- **WeightedRemainingToGoal:** heuristik ini juga **tidak dijamin admissible**. Penggunaan bobot γ dan δ membuat estimasi semakin fleksibel, tetapi

sekaligus meningkatkan risiko overestimate. Heuristik ini lebih cocok dipakai sebagai heuristik *informed* untuk mempercepat pencarian, bukan untuk menjamin optimalitas A^* .

Dengan demikian, jika tujuan utama adalah menjaga optimalitas A^* , heuristik yang paling aman dipakai adalah ManhattanToTarget dalam bentuk batas bawah cost. Sebaliknya, dua heuristik berbobot lebih cocok digunakan pada GBFS atau pada mode A^* ketika optimalitas tidak menjadi prioritas utama.

2.5 Komparasi Teoretis Algoritma Pencarian

2.5.1 Kriteria perbandingan

Perbandingan didasarkan pada beberapa kriteria teoritis yang sering dipakai dalam analisis algoritma pencarian, yaitu

- **Kompleksitas waktu:** jumlah node yang mungkin diekspansi sebagai fungsi branching factor b dan kedalaman solusi d .
- **Kompleksitas memori:** ukuran frontier dan struktur data yang disimpan (sering menjadi batas praktis utama pada pencarian graf/ruang keadaan besar).
- **Kelengkapan (completeness):** apakah algoritma menjamin menemukan solusi bila ada.
- **Optimalitas (optimality):** apakah algoritma menjamin solusi biaya minimum menurut fungsi cost yang didefinisikan.
- **Ketergantungan pada heuristik:** bagaimana kualitas/jenis heuristik mempengaruhi performa dan jaminan teori.

2.5.2 Properti Algoritma

- **Uniform-Cost Search (UCS)**
 - Kompleksitas waktu: dalam kasus terburuk, ekspansi bisa $O(b^{\lceil C^*/\epsilon \rceil})$ di mana C^* adalah biaya solusi optimal dan ϵ adalah cost min positif pada edge; secara intuitif sangat sensitif terhadap rentang dan nilai cost.
 - Kompleksitas memori: menyimpan seluruh frontier dan explored set (sering eksponensial pada d).
 - Kelengkapan: lengkap jika semua edge cost ≥ 0 dan graf tidak memuat siklus dengan cost negatif.
 - Optimalitas: menjamin optimalitas biaya jika semua edge cost non-negatif.
 - Heuristik: tidak memerlukan heuristik; tidak mendapat keuntungan dari informasi heuristik.

- **Greedy Best-First Search (GBFS)**

- Kompleksitas waktu: sangat bergantung pada kualitas heuristik; dalam analisis kasar, dapat mendekati $O(b^d)$ jika heuristik menyesatkan, tetapi sering jauh lebih sedikit node diekspansi bila heuristik informatif.
- Kompleksitas memori: menyimpan frontier (biasanya lebih kecil dari UCS/A* pada kasus yang heuristiknya sangat informatif), tetapi masih bisa besar bila heuristik buruk.
- Kelengkapan: tidak selalu lengkap pada ruang keadaan dengan branching tak terbatas; pada graf finite biasanya lengkap jika implementasi menangani revisits, tetapi tidak ada jaminan teoretis umum.
- Optimalitas: tidak optimal GBFS hanya mengejar nilai heuristik terkecil tanpa mempertimbangkan biaya yang sudah dikeluarkan.
- Heuristik: performa sangat sensitif terhadap akurasi heuristik; heuristik admissible/consistent tidak memberi jaminan optimalitas untuk GBFS.

- **A***

- Kompleksitas waktu: secara teoretis antara UCS dan GBFS; node yang diekspansi bergantung pada seberapa informatif heuristik h . Semakin mendekati biaya sisa sebenarnya, semakin sedikit node yang diekspansi.
- Kompleksitas memori: menyimpan frontier dan explored set sama seperti UCS bisa menjadi faktor pembatas utama.
- Kelengkapan: lengkap jika heuristic dan struktur graf memenuhi asumsi standar (mis. finite branching dan biaya edge non-negatif).
- Optimalitas: A* menjamin optimalitas jika heuristik *admissible* (tidak overestimate) dan biasanya memerlukan konsistensi (monotonicity) agar implementasi dengan pruning sederhana tetap benar.
- Heuristik: kualitas heuristik menentukan efektivitas; heuristik admissible tapi lebih informatif (lebih tinggi namun dari cost sebenarnya) mengurangi ekspansi signifikan.

2.5.3 Perbandingan Teoretis

- **Admissibility vs. Konsistensi:** admissibility ($h(n) \leq h^*(n)$) penting untuk optimalitas A*. Konsistensi (monotonicity) memastikan bahwa nilai f pada frontier tidak menurun dan memungkinkan implementasi tanpa perlu reprocessing node yang sudah diekspansi.
- **Pengaruh branching factor b dan kedalaman d :** banyak analisis asymptotic didasarkan pada b dan d . Ketika b besar dan d juga besar, biaya waktu dan memori tumbuh eksponensial sehingga teknik penghematan memori (IDA*, search with limited memory) perlu dipertimbangkan.



- **Trade-off waktu vs memori:** UCS dan A* umumnya menukar memori untuk waktu (menyimpan frontier besar untuk menghindari rekalkulasi), sedangkan variasi seperti IDA* menurunkan memori dengan biaya waktu tambahan.
- **Kualitas heuristik:** heuristik yang lebih informatif (lebih dekat ke biaya sisa nyata) mengurangi jumlah node yang diekspansi, namun jika heuristik meng-overestimate (non-admissible), A* kehilangan jaminan optimalitas. GBFS memilih agresivitas heuristik di atas semua, sehingga sangat cepat bila heuristik akurat tetapi sangat berisiko bila heuristik menyesatkan.

Bab 3

Implementasi

3.1 Arsitektur Program

Program solver Ice Sliding Puzzle diorganisir dengan arsitektur modular yang memisahkan tanggung jawab ke dalam berbagai modul independen. Desain ini memudahkan pemeliharaan, pengujian, dan pengembangan fitur baru.

3.1.1 Lapisan Arsitektur

Program terdiri dari beberapa lapisan logis sebagai berikut.

- **Lapisan Parsing.** Membaca file input, memvalidasi format, dan mengonversi data mentah menjadi representasi internal (Board dan State).
- **Lapisan Algoritma Pencarian.** Mengimplementasikan strategi pencarian (UCS, GBFS, A*) yang menggunakan struktur data priority queue dan menjelajahi ruang keadaan.
- **Lapisan Heuristik.** Menyediakan berbagai fungsi heuristik (ManhattanToTarget, RemainingPlusManhattan, WeightedRemainingToGoal) untuk menginformasikan pencarian.
- **Lapisan Aturan.** Mendefinisikan aturan permainan, termasuk validasi gerakan, kalkulasi biaya, dan transisi keadaan (sliding mechanics).
- **Lapisan Output.** Mengformat dan menampilkan hasil (jalur solusi, statistik, file output).
- **Lapisan GUI.** Menyediakan antarmuka visual untuk interaksi pengguna, playback solusi, dan visualisasi board.

3.1.2 Alur Eksekusi

Alur umum program adalah sebagai berikut.

1. **Parsing.** Parser membaca file input dan membuat object Board dengan initial state.

2. **Validasi.** Validator memeriksa keaslian input (dimensi, karakter, target keadaan valid).
3. **Inisialisasi Pencarian.** Search object diinisialisasi dengan algoritma pilihan dan heuristik.
4. **Pencarian.** Algoritma pencarian menjalankan `solve()` dan mengekspansi node hingga menemukan solusi atau kehabisan resource.
5. **Rekonstruksi Jalur.** Dari node tujuan, trace kembali ke root untuk mendapatkan urutan gerakan.
6. **Output.** Output formatter mencetak solusi, statistik (node expanded, waktu, biaya), dan playback command.
7. **GUI.** Jika aktif, GUI menampilkan visualisasi board dan memungkinkan pengguna playback solusi secara interaktif.

3.1.3 Desain Object-Oriented Programming (OOP)

Program solver dibangun menggunakan paradigma Object-Oriented Programming (OOP) yang memanfaatkan konsep enkapsulasi, abstraksi, dan modularitas untuk mengorganisir logika kompleks ke dalam unit-unit yang terkelola dengan baik.

Konsep OOP dalam Arsitektur

Setiap modul dalam program direpresentasikan sebagai **class** (kelas) dalam C++ yang mengenkapsulasi data (atribut/member variable) dan fungsi operasi (metode/member function). Prinsip-prinsip OOP yang diterapkan adalah

- **Enkapsulasi (Encapsulation):** Data dan fungsi yang terkait dikelompokkan dalam satu class. Akses ke data dikendalikan melalui visibility modifier (`private`, `public`, `protected`), memastikan bahwa state objek hanya dapat diubah melalui interface terdefinisi. Contoh: Class Board menyimpan grid dan checkpoint sebagai `private` members, dan hanya menyediakan method `public` seperti `isGoal()`, `getState()` untuk akses yang terkontrol.
- **Abstraksi (Abstraction):** Kompleksitas internal disembunyikan di balik interface yang sederhana. Pengguna module tidak perlu tahu detail implementasi. Contoh: Class Search menyembunyikan detail struktur internal frontier (`priority queue`) dan hanya mengekspos method `solve()` yang mengembalikan solusi.
- **Modularitas (Modularity):** Setiap class memiliki tanggung jawab tunggal dan jelas (Single Responsibility Principle). Hubungan antar class didefinisikan dengan jelas melalui dependency. Contoh: Board hanya merepresentasikan state; Rules hanya menangani mekanik permainan; Search hanya menangani strategi pencarian.

- **Reusability:** Classes dirancang untuk dapat digunakan kembali dalam konteks berbeda. Contoh: Class Board dapat digunakan oleh Search, Validator, Output, dan GUI tanpa modifikasi.

Struktur Class dan Member

Setiap modul utama diwakili oleh satu atau lebih class dengan struktur sebagai berikut.

- **Member Variable (Atribut):** Data yang menyimpan state objek. Contoh:
 - Board: `grid[][], rows, cols, startPos, goalPos, checkpoints[]`
 - Search: `frontier` (priority queue), `bestCost` (map), `explored` (set), `algorithm_type`, `heuristic_type`
 - Rules: `board` (reference to Board), `tileWeights` (map of costs)
 - Heuristic: Static methods tidak menyimpan state, hanya fungsi estimasi pure
- **Member Function (Metode):** Fungsi yang beroperasi pada state objek atau menjalankan logika spesifik modul. Contoh:
 - Board: `isGoal()`, `getState()`, `getActorPos()`, `getCheckpoints()`, `setActorPos()`
 - Search: `solve()`, `computePriority()`, `backtrack()`, `expandNode()`
 - Rules: `applyMove()`, `isValidMove()`, `calculateCost()`
 - Heuristic: `estimate()` (static), dengan overload untuk berbagai tipe heuristik
- **Constructor & Destructor:** Constructor menginisialisasi member variable dengan nilai default atau parameter yang diberikan. Destructor membersihkan resource yang dialokasikan dinamis (jika ada). Contoh:
 - `Board(int rows, int cols)`: Inisialisasi grid kosong
 - `Search(SearchType type, HeuristicType htype)`: Inisialisasi frontier dan parameter pencarian
- **Getter & Setter:** Method yang memberikan akses controlled ke private member. Getter mengembalikan nilai member; setter mengubah nilai dengan validasi opsional. Contoh: `getRows()`, `getCols()`, `setHeuristic(HeuristicType type)`.

Interaksi Antar Object

Program berjalan melalui serangkaian interaksi antar object sebagai berikut.

1. Parser menciptakan object Board dari file input: `Board board = parser.parseFiles(filename);`

2. Validator menerima reference ke Board dan memeriksa validitas: `validator.validate(board);`
3. Search object diinisialisasi dengan reference ke Board dan tipe algoritma: `Search searcher(board, SearchType::ASTAR, HeuristicType::Manhattan);`
4. Dalam `Search::solve()`, method `Rules::applyMove()` dipanggil untuk setiap successor:
`Outcome out = rules.applyMove(board, pos, dir);`
5. Heuristik digunakan dalam `computePriority()`:
`int h = Heuristic::estimate(board, pos, ...);`
6. Hasil akhir dikemas dalam object Output yang berisi solusi dan statistik:
`Output result(solution_path, expanded_count, total_cost);`
7. Jika GUI aktif, object GUI menggunakan Playback object untuk animasi:
`Playback playback(solution_path, board); playback.step();`

3.2 Direktori Program

3.2.1 Penjelasan Direktori Utama

- **src/**: Berisi seluruh source code C++ yang terbagi ke dalam core modules (board, search, heuristic, dll) dan dua entry point utama:
 - `main.cpp`: Entry point CLI solver.
 - `gui_main.cpp`: Entry point GUI solver (menggunakan raylib).
- **src/include/**: Mirror struktur `src/core`, berisi header files (.hpp) dengan deklarasi class dan interface untuk setiap modul.
- **bin/**: Direktori output untuk executable:
 - `solver`: Binary CLI yang dijalankan dari command line.
 - `solver_gui`: Binary GUI yang membuka window interaktif (raylib).
- **build/**: Direktori temporary untuk build artifacts:
 - `build/obj/`: Object files (.o) terorganisir per modul.
 - `build/obj_cli/`: Object files khusus untuk CLI build.
 - `build/obj_gui/`: Object files khusus untuk GUI build (termasuk raylib linking).
- **doc/**: Laporan teknis dalam LaTeX:
 - `main.tex`: File root LaTeX.
 - `sections/`: Bab-bab laporan (01-pendahuluan, 02-dasar-teori, 03-implementasi, dll).

- `public/`: Assets (gambar, logo).
- `dafpus.bib`: Bibliography (referensi akademik).
- `test/`: Direktori test cases berisi file input (.txt) untuk berbagai skenario puzzle (ukuran 5x5, 6x7, 8x8, dll).
- `external/vcpkg/`: Package manager dan dependency cache untuk library eksternal (raylib, eigen, dll).
- **Makefile**: Script build yang mengompilasi kedua target (CLI dan GUI) dengan flags dan dependency management.
- **vcpkg.json**: Manifest yang mendeskripsikan dependencies proyek (versi raylib, compiler triplet, dll).

3.3 Modul-Modul Program

3.3.1 Modul Board

Peran: Merepresentasikan state papan puzzle dengan menyimpan grid tile, dimensi, posisi aktor, dan informasi checkpoint. Board bertindak sebagai *data model* pusat yang digunakan oleh semua modul lain.

Atribut Utama:

- `rows_`, `cols_`: Dimensi board
- `tiles_`: 2D vector tile (Path, Wall, Lava, Start, Goal, Checkpoint)
- `costs_`: 2D vector cost setiap tile
- `start_`, `goal_`: Posisi start dan goal
- `maxCheckpoint_`, `checkpoints_`: Urutan checkpoint yang harus dikunjungi

Code Snippet:

Board Class Header

```
class Board {
public:
    Board() = default;
    void init(int rows, int cols);

    int rows() const;
    int cols() const;
    const Position& start() const;
    const Position& goal() const;
    int maxCheckpoint() const;

    void setTile(int row, int col, const Tile& tile);
    void setCost(int row, int col, int cost);

private:
    int rows_, cols_;
    std::vector<std::vector<Tile>> tiles_;
    std::vector<std::vector<int>> costs_;
    Position start_, goal_;
    int maxCheckpoint_;
    std::vector<Position> checkpoints_;
};
```

3.3.2 Modul Search

Peran: Mengimplementasikan strategi pencarian (UCS, GBFS, A*). Modul ini mengelola frontier (priority queue), tracking bestCost untuk pruning, dan merekonstruksi solusi dari node tujuan.

Metode Utama:

- **solve():** Main algorithm loop yang mengeluarkan node, cek goal, generate successors
- **computePriority():** Hitung $f = g + h$ sesuai algoritma pilihan
- **backtrack():** Rekonstruksi jalur dari goal ke start

Code Snippet:

Search Class Header

```
enum class SearchAlgorithm { UCS, GBFS, AStar };

struct SearchConfig {
    SearchAlgorithm algorithm = SearchAlgorithm::UCS;
    HeuristicType heuristic = HeuristicType::Manhattan;
};

class Search {
public:
    SearchResult solve(const Board& board,
                      const Rules& rules,
                      const SearchConfig& config) const;
private:
    char dirToChar(MoveDir dir) const;
    bool isGoalState(const Board& board,
                    const Position& pos,
                    int nextCheckpoint) const;
};
```

3.3.3 Modul Rules

Peran: Mengimplementasikan mekanik permainan (sliding mechanics), validasi gerakan, dan kalkulasi biaya. Modul ini bertanggung jawab untuk transisi state sesuai aturan puzzle.

Metode Utama:

- **applyMove():** Slide aktor ke arah tertentu, return **MoveOutcome** dengan posisi akhir, biaya, checkpoint status
- **isInside():** Validasi posisi dalam bounds
- **tileCost():** Ambil cost tile tertentu

Code Snippet:

Rules Class Header

```
enum class MoveDir { Up, Down, Left, Right };

struct MoveOutcome {
    Position end;
    bool hitWall, outOfBounds, hitLava;
    bool valid;
    int nextCheckpoint, moveCost;
};

class Rules {
public:
    MoveOutcome applyMove(const Board& board,
                          const Position& start,
                          int checkpointIndex,
                          MoveDir dir) const;
private:
    bool isInside(const Board& board,
                  int row, int col) const;
    Position step(Position pos, MoveDir dir) const;
    int tileCost(const Board& board,
                  int row, int col) const;
};
```

3.3.4 Modul Heuristic

Peran: Menyediakan fungsi estimasi untuk menginformasikan pencarian. Menawarkan tiga tipe heuristik untuk mengestimasi biaya sisa dari state saat ini ke goal.

Tipe Heuristik:

- **ManhattanToTarget:** Manhattan distance $\times$ cost minimum tile
- **RemainingPlusManhattan:** $\alpha \cdot r(n) + \beta \cdot \text{manhattan}$
- **WeightedRemainingToGoal:** $\gamma \cdot r(n) + \delta \cdot \text{manhattan to goal}$

Code Snippet:

Heuristic Class Header

```
enum class HeuristicType {
    ManhattanToTarget,
    RemainingPlusManhattan,
    WeightedRemainingToGoal
};

class Heuristic {
public:
    int estimate(const Board& board,
                const Position& pos,
                int nextCheckpoint,
                HeuristicType type) const;
private:
    int manhattan(const Position& a,
                  const Position& b) const;
    Position resolveTarget(const Board& board,
                           int nextCheckpoint) const;
    int remainingCheckpoints(const Board& board,
                             int nextCheckpoint) const;
};
```

3.3.5 Modul Parser

Peran: Membaca file input (.txt) dan mengkonversi format ASCII ke struktur data internal RawInput. Parser bertanggung jawab untuk ekstraksi data awal sebelum validasi.

Metode Utama:

- `parseFile()`: Baca file, ekstrak map, dimensi, dan cost matrix

Code Snippet:

Parser Class Header

```
class RawInput {
public:
    int rows() const;
    int cols() const;
    const std::vector<std::string>& mapLines() const;
    const std::vector<std::vector<int>>& costs() const;
    void setDimensions(int rows, int cols);
    void addMapLine(const std::string& line);
private:
    int rows_, cols_;
    std::vector<std::string> mapLines_;
    std::vector<std::vector<int>> costs_;
};

class Parser {
public:
    RawInput parseFile(const std::string& path) const;
};
```

3.3.6 Modul Validator

Peran: Memvalidasi integritas input sebelum pencarian dimulai. Memeriksa dimensi, karakter, keberadaan start/goal, dan konsistensi checkpoint sequence. Throw exception jika invalid.

Metode Utama:

- `validate()`: Validasi `RawInput`, return `Board` object jika valid

Code Snippet:

Validator Class Header

```
class Validator {
public:
    Board validate(const RawInput& raw) const;
    // Throws ValidationException jika invalid
};
```

3.3.7 Modul Output

Peran: Mengformat dan menampilkan hasil solusi dalam berbagai format (board visual, jalur langkah demi langkah, file output, statistik). Modul ini menangani all output operations.

Metode Utama:

- `renderBoard()`: Render board ASCII dengan posisi aktor saat ini

- `printSolutionSteps()`: Print jalur solusi step-by-step
- `writeSolutionFile()`: Tulis solusi ke file output
- `writeIterationFile()`: Tulis iteration log untuk analisis

Code Snippet:

OutputRenderer Class Header

```
class OutputRenderer {
public:
    std::vector<std::string> renderBoard(
        const Board& board,
        const Position& actor,
        int nextCheckpoint) const;
    void printBoard(
        const std::vector<std::string>& lines) const;
    void printSolutionSteps(
        const Board& board,
        const std::vector<Position>& positions,
        const std::vector<int>& checkpoints,
        const std::string& moves) const;
    bool writeSolutionFile(
        const std::string& path,
        const Board& board,
        const SearchResult& result) const;
};
```

3.3.8 Modul Playback

Peran: Memutar kembali solusi secara step-by-step untuk visualisasi dan verifikasi. Digunakan oleh GUI untuk animasi dan oleh CLI untuk interactive replay.

Metode Utama:

- `run()`: Eksekusi solusi dengan render setiap step, menunggu input pengguna atau waktu delay

Code Snippet:

Playback Class Header

```
class Playback {
public:
    void run(const Board& board,
        const std::vector<Position>& positions,
        const std::vector<int>& checkpoints,
        const OutputRenderer& renderer,
        size_t startIndex) const;
};
```

3.3.9 Modul Exception

Peran: Menyediakan custom exception classes untuk error handling yang terstruktur dan informatif. Memungkinkan program menangkap dan melaporkan error dengan context yang jelas.

Jenis Exception:

- **AppException:** Base exception class
- **ParseException:** Error saat parsing file
- **ValidationException:** Error saat validasi input
- **IOException:** Error saat file I/O operations

Code Snippet:

Exception Classes Header

```
enum class ExceptionType { Parse, Validation, IO };

class AppException : public std::runtime_error {
public:
    AppException(ExceptionType type,
                  const std::string& message);
    ExceptionType type() const;
private:
    ExceptionType type_;
};

class ParseException : public AppException {
public:
    explicit ParseException(
        const std::string& message);
};

class ValidationException : public AppException {
public:
    explicit ValidationException(
        const std::string& message);
};
```

3.3.10 Dependency Antar Modul

Modul-modul di atas saling bergantung dalam graf berikut:

- **Search** $\leftarrow$ Board, Rules, Heuristic, Output
- **Rules** $\leftarrow$ Board

- **Heuristic** $\leftarrow$ Board
- **Parser** $\rightarrow$ RawInput
- **Validator** $\leftarrow$ RawInput, Board, Exception
- **Output** $\leftarrow$ Board, Search (SearchResult)
- **Playback** $\leftarrow$ Board, Rules, Output
- **Exception:** Digunakan di Parser, Validator, dan modul lainnya untuk error handling

Desain modular ini memastikan setiap modul dapat dikembangkan, ditest, dan dioptimasi secara independen, sambil tetap terintegrasi dengan sempurna dalam sistem keseluruhan.

Bab 4

Eksperimen

- 4.1 Testing Kasus Dasar
- 4.2 Testing *Edge Cases*
- 4.3 *Post Mortem Analysis*

Bab 5

Penutup

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 25 Maret 2026

Muhammad Aufar Rizqi Kusuma
13524061

Bab 6

Lampiran

6.1 Tautan GitHub

Link GitHub

6.2 Checklist Tupil

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	☐	☐
2	Program berhasil dijalankan	☐	☐
3	Solusi benar dan mematuhi aturan permainan	☐	☐
4	Dapat membaca <code>.txt</code> dan menyimpan solusi ke <code>.txt</code>	☐	☐
5	Memiliki GUI	☐	☐
6	Ada algoritma pathfinding alternatif	☐	☐
7	Ada dua heuristik alternatif tambahan	☐	☐

Bibliografi

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson, 2010.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.